
Résolution par différences finies de l'équation de la chaleur en 2D

Fortran90 Seatech - MOCA

Réalisé par : Thomas DE GUZMAN DE SAINT NICOLAS
Enseignant encadrant : Mr F. GOLAY, enseignant
Date de rendu : 13 Octobre 2025

Table des matières

1	Introduction	2
1.1	Équation différentielle	3
1.2	Schéma des différences finies	4
2	Implémentation du problème en Fortran90	6
2.1	Structure générale du programme	6
2.2	Les sous-programmes principaux	6
3	Application à notre problème	7
3.1	Cas général	7
3.2	Premier cas : $N_x = N_y = 3$	9
3.3	Deuxieme cas : $N_x = N_y = 15, 31$ et 63	11
3.4	Troisieme cas : $N_x = N_y = 100$	12
4	Calcul de l'erreur	13
4.1	Définition	13
4.2	Résultats pour $N_x = N_y = 3$ et interpretation	13
4.3	Résultats pour $N_x = N_y = 15, 31$ et 63 et interpretation	14
4.4	Résultats pour $N_x = N_y = 100$ et interpretation	15
4.5	Validation du code	15
5	Conclusion	17

1 Introduction

L'objectif de ce travail pratique est de résoudre numériquement l'équation de la chaleur à l'état stationnaire en deux dimensions à l'aide de la méthode des différences finies.

Ce problème illustre la discrétisation du Laplacien sur une plaque rectangulaire soumise à des conditions aux limites de Dirichlet. Ce TP permet également de manipuler la structure matricielle issue du schéma à cinq points et de mettre en œuvre la résolution numérique du système linéaire associé en codant en **Fortran90**.

Dans notre problème, trois côtés de la plaque sont maintenus à la température $T = 0^{\circ}C$, tandis que le quatrième est imposé à $T = 100^{\circ}C$. On cherche ainsi la température $T(x, y)$ au sein du domaine en régime permanent.

1.1 Équation différentielle

On cherche à résoudre l'équation de la chaleur à l'état stationnaire sur un domaine rectangulaire de taille '

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0.$$

Il s'agit de la forme stationnaire de l'équation de propagation de la chaleur. Elle provient du bilan d'énergie sur un petit élément infinitésimal du domaine étudié (ici une plaque) avec les hypothèses suivantes :

- régime permanent (la température ne dépend pas du temps) ;
- matériau homogène et isotrope, de conductivité thermique constante k ;
- absence de source interne de chaleur ($q_v = 0$).

Donc en partant de l'équation de Fourier généralisée :

$$\nabla \cdot (k \nabla T) + q_v = 0,$$

et en supposant k constant et $q_v = 0$, on obtient directement l'équation que l'on cherche à résoudre :

$$k \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) = 0 \quad \implies \quad \Delta T(x, y) = 0 \quad \text{sur} \quad (x, y) \in]0, a[\times]0, b[$$

avec les conditions aux limites (de type **Dirichlet**) suivantes :

$$\left\{ \begin{array}{ll} T(0, y) = 0, & (\text{bord gauche froid}) \\ T(a, y) = 0, & (\text{bord droit froid}) \\ T(x, 0) = 0, & (\text{bord inférieur froid}) \\ T(x, b) = 100, & (\text{bord supérieur chaud}) \end{array} \right.$$

1.2 Schéma des différences finies

A l'aide de la méthode des différences finies, on peut discrétiser notre problème. En effet, cette méthode consiste à approximer les dérivées partielles par des différences entre les valeurs de la fonction sur le maillage. Le développement limité de Taylor autour d'un point intérieur (x_i, y_j) permet d'obtenir ces approximations.

Ainsi, on note :

$$x_i = i h_x, \quad y_j = j h_y,$$

où h_x et h_y sont les pas de discrétisation selon les directions x et y .

Développement selon x :

Pour $y = y_j$ fixé, on développe $T(\cdot, y_j)$ en x_i et on obtient :

$$\begin{aligned} T(x_{i+1}, y_j) &= T(x_i, y_j) + h_x \frac{\partial T}{\partial x}(x_i, y_j) + \frac{h_x^2}{2} \frac{\partial^2 T}{\partial x^2}(x_i, y_j) + \frac{h_x^3}{6} \frac{\partial^3 T}{\partial x^3}(x_i, y_j) + \mathcal{O}(h_x^4), \\ T(x_{i-1}, y_j) &= T(x_i, y_j) - h_x \frac{\partial T}{\partial x}(x_i, y_j) + \frac{h_x^2}{2} \frac{\partial^2 T}{\partial x^2}(x_i, y_j) - \frac{h_x^3}{6} \frac{\partial^3 T}{\partial x^3}(x_i, y_j) + \mathcal{O}(h_x^4). \end{aligned}$$

En additionnant ces deux développements, on élimine les termes en dérivées premières et troisièmes, ce qui donne :

$$T(x_{i+1}, y_j) + T(x_{i-1}, y_j) = 2T(x_i, y_j) + h_x^2 \frac{\partial^2 T}{\partial x^2}(x_i, y_j) + \mathcal{O}(h_x^4).$$

On obtient alors l'approximation suivante :

$$\frac{\partial^2 T}{\partial x^2}(x_i, y_j) = \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{h_x^2} + \mathcal{O}(h_x^2).$$

Développement selon y :

Pour $x = x_i$ fixé, on développe $T(x_i, \cdot)$ en y_j et on obtient :

$$\begin{aligned} T(x_i, y_{j+1}) &= T(x_i, y_j) + h_y \frac{\partial T}{\partial y}(x_i, y_j) + \frac{h_y^2}{2} \frac{\partial^2 T}{\partial y^2}(x_i, y_j) + \frac{h_y^3}{6} \frac{\partial^3 T}{\partial y^3}(x_i, y_j) + \mathcal{O}(h_y^4), \\ T(x_i, y_{j-1}) &= T(x_i, y_j) - h_y \frac{\partial T}{\partial y}(x_i, y_j) + \frac{h_y^2}{2} \frac{\partial^2 T}{\partial y^2}(x_i, y_j) - \frac{h_y^3}{6} \frac{\partial^3 T}{\partial y^3}(x_i, y_j) + \mathcal{O}(h_y^4). \end{aligned}$$

En additionnant ces deux développements, on élimine les termes en dérivées premières et troisièmes, ce qui donne :

$$T(x_i, y_{j+1}) + T(x_i, y_{j-1}) = 2T(x_i, y_j) + h_y^2 \frac{\partial^2 T}{\partial y^2}(x_i, y_j) + \mathcal{O}(h_y^4).$$

On obtient alors l'approximation suivante :

$$\frac{\partial^2 T}{\partial y^2}(x_i, y_j) = \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{h_y^2} + \mathcal{O}(h_y^2).$$

Dans l'équation initiale :

Or, l'équation différentielle initiale est :

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0.$$

Ainsi, en remplaçant chaque dérivée seconde par son approximation discrète, on obtient :

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{h_x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{h_y^2} = 0 + \mathcal{O}(h_x^2 + h_y^2).$$

Ainsi, le schéma obtenu est d'ordre 2 en espace. Cette relation relie la température en un point intérieur $T_{i,j}$ à celles de ses quatre voisins directs.

$$\boxed{\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{h_x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{h_y^2} = 0}$$

avec :

$$h_x = \frac{a}{N_x + 1}, \quad h_y = \frac{b}{N_y + 1}.$$

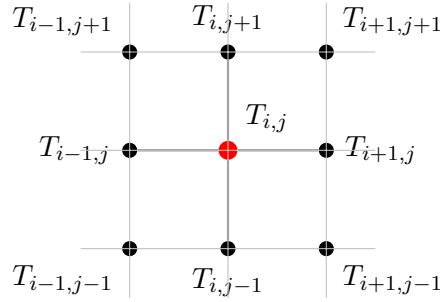


FIGURE 1 – Illustration du point $T_{i,j}$ et de ses quatre voisins.

Cette équation conduit à un système linéaire $A\mathbf{T} = \mathbf{F}$, où A est une matrice tridiagonale par blocs représentant le couplage entre les points du maillage.

2 Implémentation du problème en Fortran90

2.1 Structure générale du programme

Le programme TP3.f90 est intégralement codé en Fortran90, langage choisi pour ses performances en calcul scientifique et sa capacité à manipuler efficacement de grandes structures matricielles. L'organisation du code suit une logique claire et modulaire grâce à l'utilisation de sous-programmes (CONTAINS).

La structure générale du programme est la suivante :

1. Initialisation des dimensions (x, y) et du maillage (N_x, N_y) .
2. Calcul des pas de maillage $h_x = a/(N_x + 1)$ et $h_y = b/(N_y + 1)$.
3. Création de $(A, B, C, F, U, T_{\text{app}}, T_{\text{exact}})$.
4. Construction de la matrice globale A à partir des blocs B et C et du vecteur second membre F issu des conditions aux limites.
5. Utilisation de la sous-routine SOLP, qui met en œuvre une élimination de Gauss pour résoudre le système $A.U = F$.
6. Calcul de la solution exacte via la série de Fourier, en sommant les termes jusqu'à une précision fixée.
7. Calcul des erreurs entre la solution approchée T_{app} et la solution exacte T_{exacte} .
8. Création de fichiers .dat au format Tecplot pour visualiser avec le logiciel VisIt.

2.2 Les sous-programmes principaux

- SOLP : résolution du système $AU = F$ par la méthode de Gauss.
- Texact_point : calcule la température analytique $T(x, y)$ en chaque point à l'aide d'une série infinie tronquée.
- erreur12 : évalue la norme L^2 et l'erreur maximale entre les champs approché et exact.
- write_tecplot : exporte les champs de température dans un fichier compatible avec Tecplot/VisIt.

Le programme complet ainsi que les sous-programmes sont fournis en Annexe.

3 Application à notre problème

3.1 Cas général

La discrétisation de l'équation de la chaleur sur un maillage régulier conduit alors au système suivant :

$$AU = F$$

où A est une matrice tridiagonale par blocs construite à partir des matrices B et C suivantes :

$$A = \begin{bmatrix} B & C & 0 & 0 & \cdots & 0 \\ C & B & C & 0 & \cdots & 0 \\ 0 & C & B & C & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & C & B & C \\ 0 & \cdots & 0 & 0 & C & B \end{bmatrix}$$

avec :

$$B = \begin{bmatrix} 2\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right) & -\frac{1}{h_x^2} & 0 & \cdots & 0 \\ -\frac{1}{h_x^2} & 2\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right) & \ddots & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -\frac{1}{h_x^2} \\ 0 & \cdots & 0 & -\frac{1}{h_x^2} & 2\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right) \end{bmatrix}, \quad C = \begin{bmatrix} -\frac{1}{h_y^2} & 0 & \cdots & 0 \\ 0 & -\frac{1}{h_y^2} & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & -\frac{1}{h_y^2} \end{bmatrix}$$

Ainsi, la matrice A a une structure tridiagonale par blocs, chaque bloc B représentant le couplage en x et les blocs C le couplage entre les lignes suivant y .

De plus, on note $\mathbf{U}$ le vecteur colonne regroupant les températures inconnues à l'intérieur du domaine, selon la numérotation des nœuds du maillage, et $\mathbf{F}$ le vecteur second membre qui contient les conditions aux limites, ici dues au bord supérieur à $T = 100^\circ C$ et aux autres côtés à $T = 0^\circ C$.

Pour un maillage (N_x, N_y) , on obtient alors les vecteurs $\mathbf{U}$ et $\mathbf{F}$ de taille $N_x \times N_y$:

$$\mathbf{U} = \begin{bmatrix} T_{1,1} \\ T_{2,1} \\ \vdots \\ T_{N_x,1} \\ T_{1,2} \\ T_{2,2} \\ \vdots \\ T_{N_x,2} \\ \vdots \\ T_{1,N_y} \\ T_{2,N_y} \\ \vdots \\ T_{N_x,N_y} \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \vdots \\ \frac{100}{h_y^2} \\ \frac{100}{h_y^2} \\ \vdots \\ \frac{100}{h_y^2} \end{bmatrix}$$

Ainsi, les températures sont empilées ligne par ligne.

De plus, seules les N_x dernières valeurs de la matrice $\mathbf{F}$ sont non nulles.

3.2 Premier cas : $N_x = N_y = 3$

On se place dans le cas particulaire où $N_x = N_y = 3$.

On obtient alors les matrices suivantes :

$$A = \begin{bmatrix} B & C & 0 \\ C & B & C \\ 0 & C & B \end{bmatrix}$$

avec :

$$B = \begin{bmatrix} 2\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right) & -\frac{1}{h_x^2} & 0 \\ -\frac{1}{h_x^2} & 2\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right) & -\frac{1}{h_x^2} \\ 0 & -\frac{1}{h_x^2} & 2\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right) \end{bmatrix}, \quad C = \begin{bmatrix} -\frac{1}{h_y^2} & 0 & 0 \\ 0 & -\frac{1}{h_y^2} & 0 \\ 0 & 0 & -\frac{1}{h_y^2} \end{bmatrix}$$

et :

$$\mathbf{U} = \begin{bmatrix} T_{1,1} \\ T_{2,1} \\ T_{3,1} \\ T_{1,2} \\ T_{2,2} \\ T_{3,2} \\ T_{1,3} \\ T_{2,3} \\ T_{3,3} \end{bmatrix}, \quad \mathbf{F} = \frac{1}{h^2} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 100 \\ 100 \\ 100 \end{bmatrix}.$$

Ce qui conduit aux résultats suivants :

$$T_{\text{app}} = \langle 7.14, 9.82, 7.14, 18.75, 25.00, 18.75, 42.86, 52.68, 42.86 \rangle$$

$$T_{\text{exact}} = \langle 6.80, 9.54, 6.80, 18.20, 25.00, 18.20, 43.20, 54.05, 43.20 \rangle$$

Une fois ces résultats obtenus, nous avons exporté au format **.Tecplot** les champs de température T_{app} et T_{exact} afin d'être visualisés dans le logiciel **VisIt**. Ce format permet de représenter la température en chaque nœud du maillage et d'en observer la répartition spatiale.

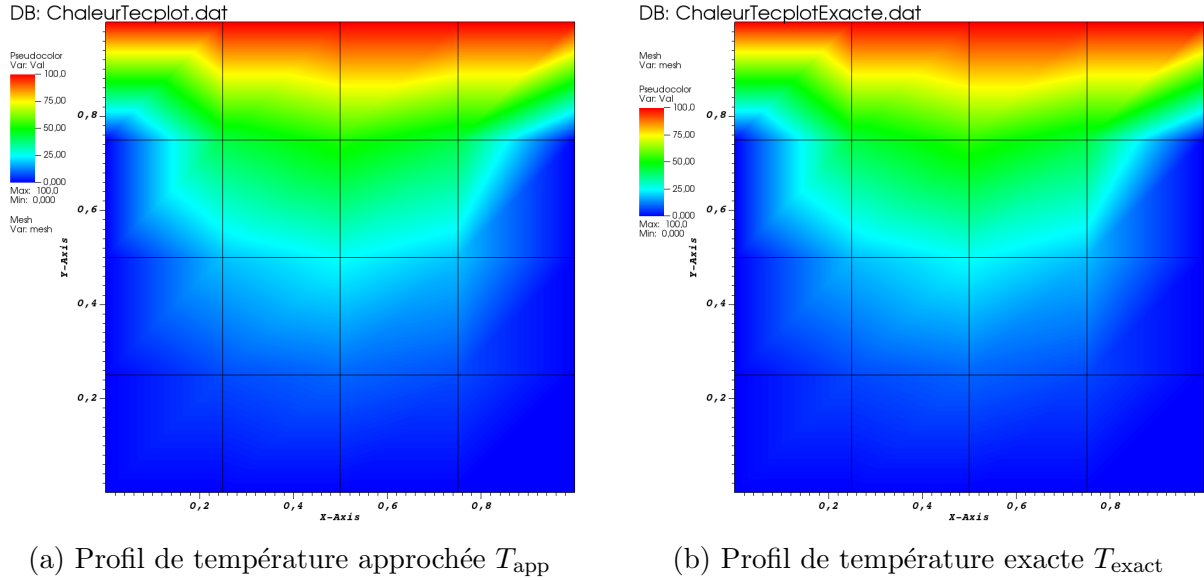


FIGURE 2 – Visualisation des champs de température sous VisIt.

Ainsi, l'utilisation de VisIt permet de comparer visuellement la solution approchée (figure de gauche) et la solution analytique (figure de droite).

On constate alors que la distribution de température est régulière et respecte la symétrie attendue du problème physique. En effet, la température décroît progressivement du bord chaud (100 °C) vers le bord froid (0 °C).

Cependant, on remarque que les courbes de température ne sont pas lisses et la décroissance de la température n'est pas régulière. En effet, cela est dû au faible nombre de neurds de calcul. Ainsi, lorsqu'ils sont reliés par le logiciel, des droite secante pas naturelle apparaissent.

Enfin, on peut se rendre compte que le modèle analytique n'est pas si éloigné du modèle représentant la solution approchées. De plus, les écarts visibles entre T_{app} et T_{exact} sont principalement localisés près des bords, ce qui est cohérent avec la précision d'ordre 2 du schéma de différences finies utilisé.

3.3 Deuxieme cas : $N_x = N_y = 15, 31$ et 63

On remarque qu'en augmentant progressivement la taille du maillage avec $N_x = N_y = 15, 31$ et 63 , la distribution de l'erreur devient nettement plus régulière et de plus faible amplitude que pour le cas $N = 3$. On observe en effet que les zones d'erreur maximale persistent principalement dans les coins supérieurs du domaine, où les gradients de température sont les plus importants.

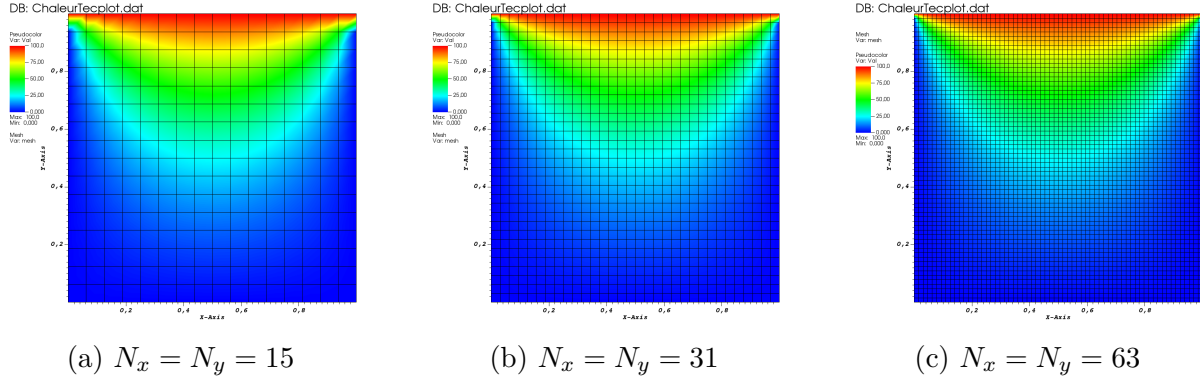


FIGURE 3 – Profil de température approchée T_{app} pour $N_x = N_y = 15, 31$ et 65 .

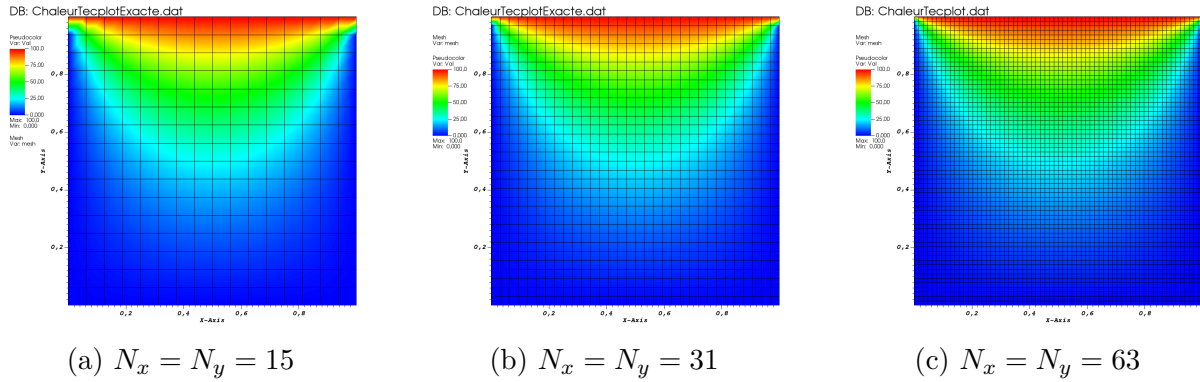


FIGURE 4 – Profil de température exacte T_{exact} pour $N_x = N_y = 15, 31$ et 65 .

Cependant, à mesure que le maillage se raffine, ces zones se réduisent et l'erreur au centre du domaine devient quasiment uniforme. Cela traduit une meilleure approximation de la solution exacte et une bonne convergence du schéma numérique. On se rapproche ainsi du comportement observé pour $N = 100$, où l'erreur devient localisée uniquement dans les zones de forts gradients et reste très faible ailleurs.

3.4 Troisième cas : $N_x = N_y = 100$

On se place alors maintenant dans le cas particulier où $N_x = N_y = 100$.

On obtient alors, grâce au logiciel VisIt :

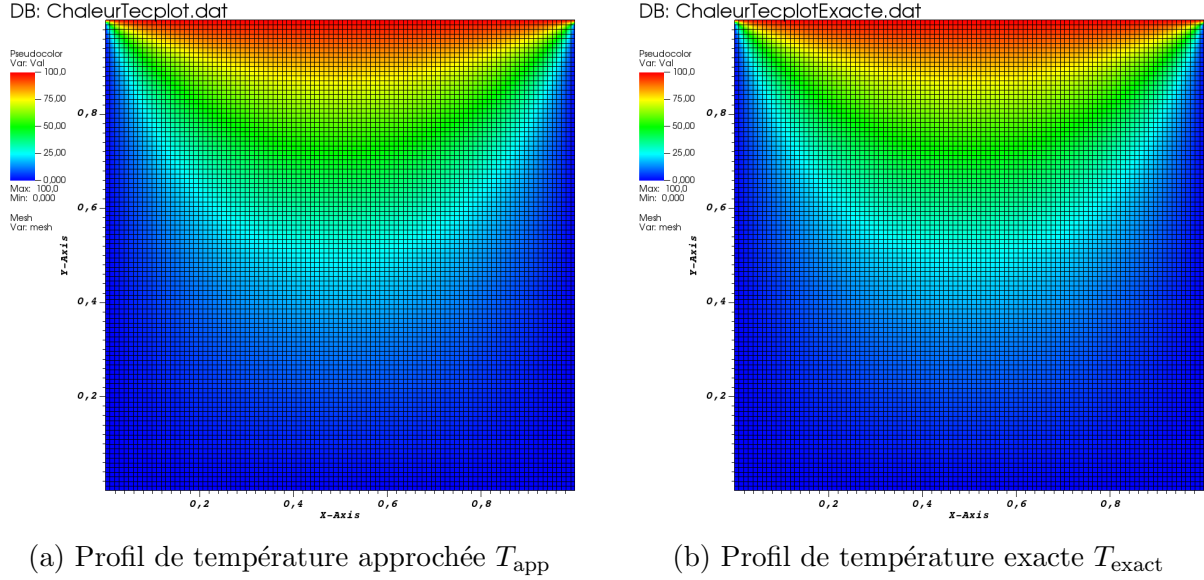


FIGURE 5 – Visualisation des champs de température sous VisIt.

On se rend ainsi compte que la température évolue bien de façon régulière avec une décroissance du bord chaud à 100 °C vers le bord froid à 0 °C.

Pour un meilleur rendu visuel, ces deux figures sans le maillage sont disponibles dans les annexes (Annexe 1a et 1b).

4 Calcul de l'erreur

4.1 Définition

L'erreur entre la solution approchée T_{app} et la solution exacte T_{exact} est évaluée selon deux normes :

$$\varepsilon_{L^2} = \sqrt{\frac{1}{N_x N_y} \sum_{i,j} (T_{\text{app}}(i,j) - T_{\text{exact}}(i,j))^2}, \quad \varepsilon_{\max} = \max_{i,j} |T_{\text{app}}(i,j) - T_{\text{exact}}(i,j)|.$$

Ces grandeurs sont calculées dans la sous-routine `erreur12`, après reconstruction des matrices complètes $T_{\text{app,full}}$ et $T_{\text{exact,full}}$.

4.2 Résultats pour $N_x = N_y = 3$ et interpretation

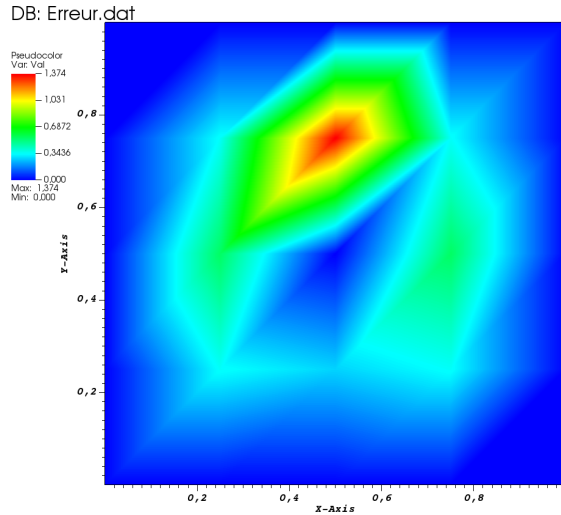


FIGURE 6 – Profil de l'erreur pour
 $N_x = N_y = 3$

Les valeurs obtenues lors de l'exécution du programme sont :

$$\varepsilon_{L^2} = 0.3489 \quad \varepsilon_{\max} = 1.3743$$

Ces valeurs sont cohérentes avec un schéma centré d'ordre 2.

On constate que pour $N_x = N_y = 3$, la carte de l'erreur paraît irrégulière. Cela s'explique par le faible nombre de points de calcul : le maillage est trop grossier, et la visualisation dans `VisIt` interpole entre peu de valeurs, ce qui donne un aspect anguleux. On relève néanmoins que l'erreur est la plus élevée près du bord chaud et au début de la dissipation, où le gradient de température reste le plus fort.

4.3 Résultats pour $N_x = N_y = 15, 31$ et 63 et interpretation

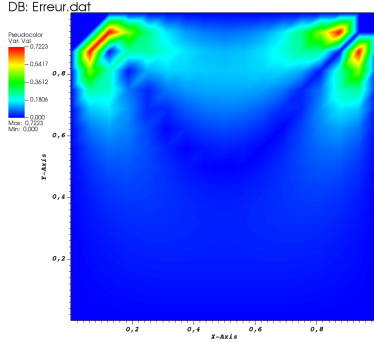


FIGURE 7 – Profil de l'erreur pour $N_x = N_y = 15$

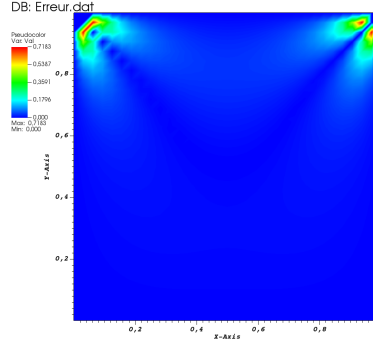


FIGURE 8 – Profil de l'erreur pour $N_x = N_y = 31$

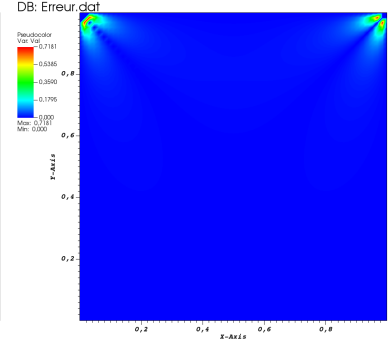


FIGURE 9 – Profil de l'erreur pour $N_x = N_y = 63$

En augmentant progressivement la taille du maillage avec $N_x = N_y = 15, 31$ et 63 , la distribution de l'erreur devient nettement plus régulière et de plus faible amplitude que pour le cas $N = 3$. On observe que les zones d'erreur maximale persistent près du bord chaud et dans les coins supérieurs du domaine, où les gradients de température sont les plus importants.

Cependant, à mesure que le maillage se raffine, ces zones se réduisent et l'erreur au centre du domaine devient quasiment uniforme. Cela traduit une meilleure approximation de la solution exacte et une bonne convergence du schéma numérique. On se rapproche ainsi du comportement observé pour $N = 100$, où l'erreur devient localisée uniquement dans les zones de forts gradients et reste très faible ailleurs.

4.4 Résultats pour $N_x = N_y = 100$ et interpretation

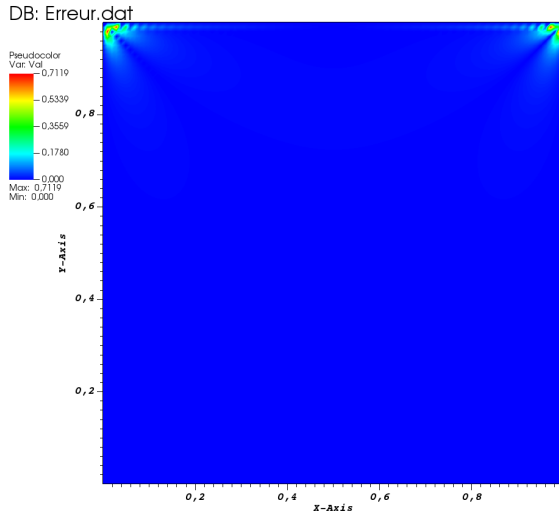


FIGURE 10 – 2-in-1 swimshort

Les valeurs obtenues lors de l'exécution du programme sont :

$$\varepsilon_{L^2} = 2.1402 \times 10^{-2} \quad \varepsilon_{\max} = 0.7118$$

Ainsi, en raffinant le maillage avec $N_x = N_y = 100$, on observe une décroissance des erreurs.

Finalement, on constate bien que, pour $N_x = N_y = 100$, l'erreur est bien maximale dans les coins supérieurs du domaine, où les gradients de température sont les plus forts. Il est donc plus difficile de calculer une température proche de la température exacte avec peu de points de discrétisation. Pour résoudre ce problème, on pourrait par exemple faire un maillage plus dense dans ces zones.

Cependant, on remarque aussi que la répartition de l'erreur est régulière au centre et sur le reste du domaine, ce qui confirme la stabilité de notre schéma.

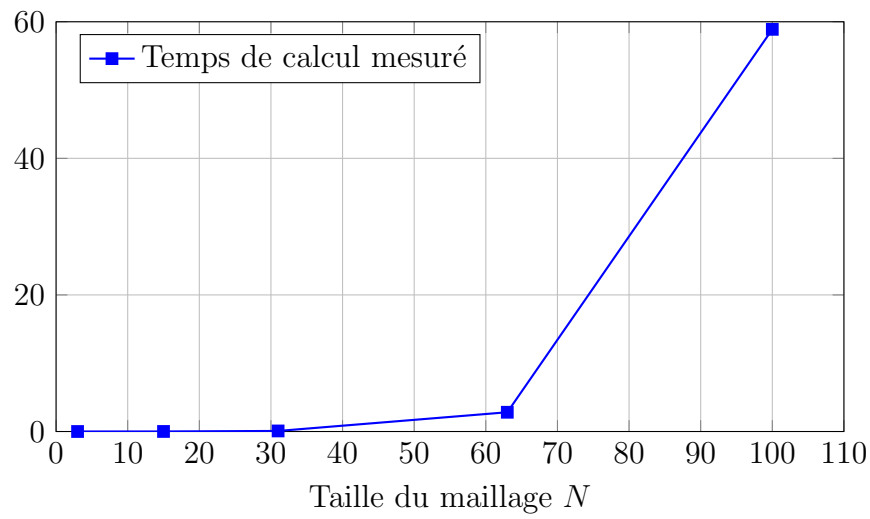
4.5 Validation du code

Ainsi, ces résultats obtenus nous permettent de valider à la fois :

- la cohérence de la construction de la matrice A ;
- la mise en œuvre correcte des conditions aux limites ;
- la précision attendue d'un schéma centré d'ordre 2.

Le code **Fortran90** développé permet ainsi de résoudre efficacement et avec précision le problème stationnaire de conduction thermique bidimensionnelle.

Évolution du temps de calcul en fonction de la taille du maillage

FIGURE 11 – Temps de calcul en fonction de $N_x = N_y = N$.

On peut remarquer que le temps d'exécution du programme augmente avec la taille du maillage N .

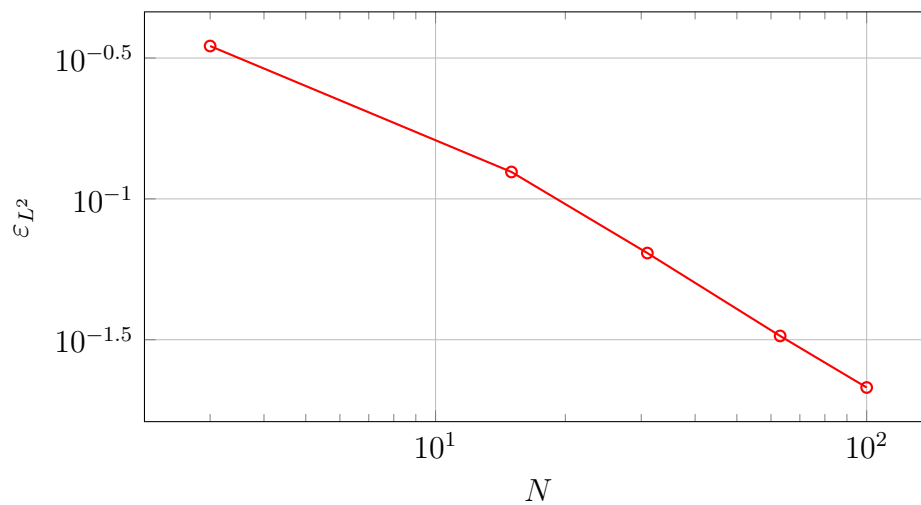
Évolution de l'erreur L^2 en fonction du raffinement du maillage

FIGURE 12 – Étude de convergence de l'erreur.

On peut remarquer que lorsque le raffinement du maillage augmente, on a une convergence de l'erreur vers zéro.

5 Conclusion

Ainsi, ce TP m'a permis de mettre en œuvre la méthode des différences finies pour résoudre l'équation de la chaleur à l'état stationnaire.

Le programme Fortran90 que j'ai développé montre la capacité de ce langage à gérer efficacement des problèmes numériques de grande taille grâce à la manipulation de tableaux et de structures matricielles. De plus, on se rend compte qu'une utilisation correcte de ce langage peut permettre de gagner un temps de calcul considérable. Cependant, on peut encore améliorer notre code en réutilisant par exemple les matrices B et C ôt que de construire entièrement la matrice A, ou on pourrait travailler avec un stockage de matices creuses (exemple : stockage CSR) pour gagner en temps de calculs.

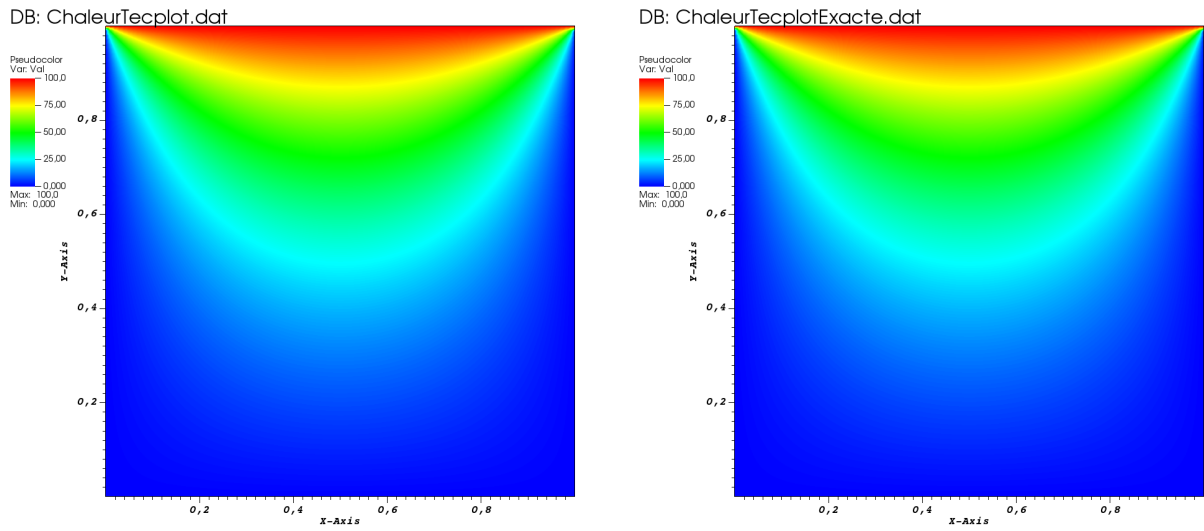
Enfin, les résultats obtenus sont en bon accord avec la solution analytique, ce qui confirme bien la justesse du schéma que nous avons employé.

Bibliographie

Références

- [1] P. Lascau, *Méthodes numériques – Résolution d'équations aux dérivées partielles par différences finies*, Notes de cours, Département MOCA, SeaTech – Université de Toulon, 2024.
- [2] S. C. Chapra et R. P. Canale, *Méthodes numériques pour ingénieurs*, 6^e édition, Pearson Éducation, 2010.
- [3] L. Mieusse, *Introduction au Fortran 90*, Université de Bordeaux, 2024. Disponible en ligne : https://www.math.u-bordeaux.fr/~lmieusse/PAGE_WEB/ENSEIGNEMENT/cours_f90.pdf.
- [4] P. Gaborit, *Initiation au Fortran*, IMT Mines Albi, 2024. Disponible en ligne : <https://perso.imt-mines-albi.fr/~gaborit/lang/Fortran.pdf>.
- [5] A. Kielbasa, *Introduction à Fortran 90*, ENSTA Paris, 2023. Disponible en ligne : <https://perso.ensta-paris.fr/~kielbasi/docs/f90.pdf>.

Annexe



(a) Profil de température approchée T_{app}

(b) Profil de température exacte T_{exact}

FIGURE 13 – Visualisation des champs de température sous VisIt pour $N_x = N_y = 100$ sans le maillage tracé.

Le programme complet utilisé pour la résolution de l'équation de la chaleur en 2D est présenté ci-dessous. Il a été rédigé en Fortran90 et commente les principales étapes : allocation dynamique, assemblage des matrices A , B , C , construction du vecteur $\mathbf{F}$, résolution du système linéaire, calcul de l'erreur et export des résultats vers le format Tecplot pour visualisation sous VisIt.

```

1  ! =====
2  !  Programme TP3 : Equation de la chaleur 2D (SeaTech - MOCA)
3  ! =====
4
5  PROGRAM TP3
6
7  implicit none
8  integer :: Nx, Ny, i, j, n, m, k
9  real(8) :: errL2, errMax
10 real(kind=8) :: x, y, hx, hy
11 real(kind=8),dimension(:,,:), allocatable :: A , B , C , Tapp_full
    , Texacte_full, er
12 real(kind=8),dimension(:), allocatable :: F, U, Tapp, Texacte
13
14 ! ----- Lecture des param tres -----
15 write(*,*) 'Valeur de x :'; read(*,*) x
16 write(*,*) 'Valeur de y :'; read(*,*) y
17 write(*,*) 'Valeur de Nx :'; read(*,*) Nx
18 write(*,*) 'Valeur de Ny :'; read(*,*) Ny
19
20 ! ----- Discr tisation -----
21 hx = x / (Nx+1)
22 hy = y / (Ny+1)
23 k = 0
24
25 ! ----- Allocation -----
26 allocate(A(Ny*Nx,Ny*Nx), B(Nx,Nx), C(Nx,Nx))
27 allocate(F(Nx*Ny), U(Nx*Ny), Tapp(Nx*Ny), Texacte(Nx*Ny))
28 allocate(Tapp_full(Nx+3,Ny+3), Texacte_full(Nx+3,Ny+3), er(Nx+3,
    Ny+3))
29
30 ! Initialisation
31 A = 0.d0; B = 0.d0; C = 0.d0
32 F = 0.d0; Tapp = 0.d0; Texacte = 0.d0
33

```

```

34 ! ===== MATRICE B =====
35 B(1,1) = -2.d0/(hx**2) - 2.d0/(hy**2)
36 B(1,2) = 1.d0/(hx**2)
37 do i=2,Nx-1
38     B(i,i-1) = 1.d0/(hx**2)
39     B(i,i)   = -2.d0/(hx**2) - 2.d0/(hy**2)
40     B(i,i+1) = 1.d0/(hx**2)
41 end do
42 B(Nx,Nx-1) = 1.d0/(hx**2)
43 B(Nx,Nx)   = -2.d0/(hx**2) - 2.d0/(hy**2)
44
45 ! ===== MATRICE C =====
46 do i=1,Nx
47     C(i,i) = 1.d0/(hy**2)
48 end do
49
50 ! ===== MATRICE A =====
51 i = 1
52 do while (i < Nx*Ny)
53     do m = 1, Nx
54         do n = 1, Nx
55             A(m+i-1, n+i-1) = B(m, n)
56         end do
57     end do
58     i = i + Nx
59 end do
60
61 i = Nx + 1
62 do while (i < Nx*Ny)
63     do m = 1, Nx
64         do n = 1, Nx
65             A(m+i-(Nx+1), n+i-1) = C(m, n)
66             A(m+i-1, n+i-(Nx+1)) = C(m, n)
67         end do
68     end do
69     i = i + Nx
70 end do
71
72
73
74

```

```

75 ! ===== VECTEUR F =====
76 do i = Nx*Ny-(Nx-1), Nx*Ny
77     F(i) = -100.d0/(hy**2)
78 end do
79
80 ! ===== RESOLUTION =====
81 call SOLP(0, A, F, 1, 1, Nx*Ny, Tapp)
82
83 ! ===== SOLUTION EXACTE =====
84 k = 1
85 do j = 1, Ny
86     do i = 1, Nx
87         call Texact_point(i*hx, j*hy, x, y, Texacte(k))
88         k = k + 1
89     end do
90 end do
91
92 ! ===== ERREUR =====
93 call erreurl2(Nx, Ny, Tapp_full, Texacte_full, errL2, errMax)
94 print *, "Erreur_L2=", errL2
95 print *, "Erreur_max=", errMax
96
97 ! ===== EXPORT TECPLOT =====
98 call write_tecplot(Nx, Ny, hx, hy, Tapp_full, 'ChaleurTecplot.dat
99     ')
100 call write_tecplot(Nx, Ny, hx, hy, Texacte_full, '
101     ChaleurTecplotExacte.dat')
102 call write_tecplot(Nx, Ny, hx, hy, er , 'Erreur.dat')
103
104 ! #####
105 ! ##### CONTAINS #####
106 ! #####
107
108 SUBROUTINE SOLP (NSYM,VKG,VFG,IKG,IFG,NEQ,VU)
109 SUBROUTINE erreurl2(Nx, Ny, Tapp_full, T_exact_full, erreur_L2,
110     erreur_max)
111 SUBROUTINE Texact_point(x, y, a, b, Texacte)
112 SUBROUTINE write_tecplot(Nx, Ny, hx, hy, Tapp_full, filename)
113
114 END PROGRAM TP3

```

Listing 1 – Programme TP3.f90 – Résolution 2D de l'équation de la chaleur

```

1  SUBROUTINE SOLP      (NSYM,VKG,VFG,IKG,IFG,NEQ,VU)
2  !===== SIC - Code de calcul E.F. - V2.31 - Module de calcul =====
3  !      Resolution d'un systeme d'equations non symetrique par la
4  !      methode de Gauss.
5  !
6  !      ENTREES
7  !          NSYM      .EQ.1  SYSTEME NON SYMETRIQUE
8  !          NEQ        NOMBRE D'EQUATIONS (.GE.2)
9  !          VKG        MATRICE K STOCKEE EN DOUBLE PRECISION (2
10         DIMENSIONS)
11         VFG          SECOND MEMBRE
12         IKG          .EQ.0  PAS DE TRIANGULARISATION
13         IFG          .EQ.1  TRIANGULARISATION DE KG
14         IFG          .EQ.0  PAS DE RESOLUTION
15         IFG          .EQ.1  RESOLUTION
16         VU           SOLUTION
17         !
18         !===== DEBUT DES DECLARATIONS =====
19         !.1----- Implicit, External
20         implicit none
21         real(kind=8)::vzero=0.d0,cl,rpiv
22         real(kind=8),allocatable,dimension(:,:)::vkg
23         real(kind=8),allocatable,dimension(:)::VFG,VU
24         integer :: NSYM,IKG,IFG,NEQ, i,j,ij,n1,ij1,is,is1,ii
25
26         !----- Triangularisation
27         IF (IKG.EQ.0.AND.IFG.EQ.0) GOTO 9999
28
29         N1=NEQ-1
30         DO 50 IS=1,N1
31             RPIV=VKG(IS,IS)
32             IF (RPIV) 10,800,10
33
34 10         IS1=IS+1
35             DO 50 II=IS1,NEQ
36                 IF(NSYM.EQ.1) CL=VKG(II,IS)
37                 IF(NSYM.NE.1) CL=VKG(IS,II)
38                 IF(CL.EQ.VZERO) GOTO 50
39                 CL=CL/RPIV
40                 IF (IFG.EQ.1) VFG(II)=VFG(II)-CL*VFG(IS)

```



```

41      IF (IKG.EQ.0) GOTO 50
42      IF (NSYM.EQ.1) GOTO 32
43      DO 30 IJ=II,NEQ
44      VKG(II,IJ)=VKG(II,IJ)-CL*VKG(IS,IJ)
45 30    VKG(IJ,II)=VKG(II,IJ)
46      GOTO 50
47 32    DO 40 IJ=IS1,NEQ
48      VKG(II,IJ)=VKG(II,IJ)-CL*VKG(IS,IJ)
49 50    CONTINUE
50
51  !-----  RESOLUTION DU SYSTEME TRIANGULAIRE
52      IF (VKG(NEQ,NEQ).NE.VZERO) GOTO 55
53      IS=NEQ
54      GOTO 800
55 55    IF (IFG.EQ.0) GOTO 9999
56      VU(NEQ)=VFG(NEQ)/VKG(NEQ,NEQ)
57      DO 70 II=1,N1
58      IS1=IS1-1
59      CL=VZERO
60      IJ1=IS1+1
61      DO 60 IJ=IJ1,NEQ
62 60    CL=CL+VKG(IS1,IJ)*VU(IJ)
63 70    VU(IS1)=(VFG(IS1)-CL)/VKG(IS1,IS1)
64
65  !-----  Fin SUBROUTINE SOLP
66      GOTO 9999
67
68  !-----  Erreurs
69 800    WRITE(*,8000) IS
70 8000   FORMAT(' * SOLP -E-RPIVNUL Pivot nul equation',I5)
71      GOTO 9999
72
73  !-----  Fin
74 9999   CONTINUE
75      RETURN
76  !=====  FIN DU MODULE SOLP
77      END SUBROUTINE SOLP

```

Listing 2 – SUBROUTINE SOLP

```

1 SUBROUTINE erreurl2(Nx, Ny, Tapp_full, T_exact_full, erreur_L2,
  erreur_max)
2
3      ! =====
4      ! ===== Declaration des variables =====
5      ! =====
6      implicit none
7
8      integer :: Nx, Ny, i, j
9      real(kind=8), dimension(:,:) :: Tapp_full, T_exact_full
10     real(kind=8) :: erreur_L2, erreur_max, diff, somme
11
12     erreur_L2 = 0.d0
13     erreur_max = 0.d0
14     somme = 0.d0
15
16     do j = 1, Ny+2
17         do i = 1, Nx+2
18             diff = Tapp_full(i,j) - T_exact_full(i,j)
19             erreur_L2 = erreur_L2 + diff*diff
20             if (abs(diff) > erreur_max) erreur_max = abs(diff)
21             somme = somme + 1.d0
22         end do
23     end do
24
25     erreur_L2 = sqrt(erreur_L2 / somme)
26
27 END SUBROUTINE erreurl2

```

Listing 3 – SUBROUTINE Erreurl2

```

1 SUBROUTINE Texact_point(x, y, a, b, Texacte)
2
3 ! ENTREES :
4 !      x      : Coordonn e x du point o  calculer la solution.
5 !      y      : Coordonn e y du point o  calculer la solution.
6 !      a      : Largeur de la plaque (dimension x).
7 !      b      : Hauteur de la plaque (dimension y).
8 !
9 ! SORTIE :
10 !      Texact  : Valeur analytique de la temp rature T(x,y) au
      point donn .
11
12
13
14      ! =====
15      ! ===== Declaration des variables =====
16      ! =====
17
18      implicit none
19      real(kind=8) :: x, y, a, b
20
21      real(kind=8) :: Texacte
22
23
24
25      integer :: n
26      real(kind=8) :: pi, term
27      real(kind=8) :: eps
28
29
30      pi = 4.d0*atan(1.d0)
31      Texacte = 0.d0
32      eps = 1.d0
33      n=0
34
35      do while(abs(eps)>1.e-10)
36          eps = (4.d0*100.d0/((2*n+1)*pi)) * &
37              sinh((2*n+1)*pi*y/a) / sinh((2*n+1)*pi*b/a) * &
38              sin((2*n+1)*pi*x/a)
39          Texacte = Texacte + eps
40          n=n+1
41      end do
42
43 END SUBROUTINE Texact_point

```

Listing 4 – SUBROUTINE TexactPoint

```

1 SUBROUTINE write_tecplot(Nx, Ny, hx, hy, Tapp_full, filename)
2
3     ! =====
4     ! ===== Declaration des variables =====
5     ! =====
6     implicit none
7
8     integer, intent(in) :: Nx, Ny
9     real(kind=8), intent(in) :: hx, hy
10    real(kind=8), dimension(:, :), intent(in) :: Tapp_full
11    character(len=*), intent(in) :: filename
12    integer :: i, j, unit
13    unit = 20
14    open(unit=unit, file=filename, status='unknown')
15    ! Header
16    write(unit,*) 'TITLE="Resolution equation de chaleur 2D"'
17    write(unit,*) 'VARIABLES="X","Y","Val"'
18    write(unit,*) 'ZONE_T="Nom de la zone" N=', (Nx+2)*(Ny+2), &
19                ',E=', (Nx+1)*(Ny+1), ',F=FEPOINT,ET=
20                QUADRILATERAL'
21
22    ! Nodal values
23    do j = 0, Ny+1
24        do i = 0, Nx+1
25            write(unit, '(3F12.6)') i*hx, j*hy, Tapp_full(i+1,j+1)
26        end do
27    end do
28
29    ! Connectivity (quadrilateral elements)
30    do j = 1, Ny+1
31        do i = 1, Nx+1
32            write(unit,*) i + (j-1)*(Nx+2), &
33                i+1 + (j-1)*(Nx+2), &
34                i+1 + j*(Nx+2), &
35                i + j*(Nx+2)
36        end do
37    end do
38    close(unit)
39
40 END SUBROUTINE write_tecplot

```

Listing 5 – SUBROUTINE WriteTecplot

```
1
2 real(8) :: t1, t2
3 call cpu_time(t1)
4
5 ! === Ton programme principal ===
6 ! Assemblage, r solution, etc.
7
8 call cpu_time(t2)
9 print *, "Temps de calcul=", t2 - t1, " secondes"
```

Listing 6 – Calcul et affichage du temps d'exécution